Date:10/10/2026

CO1-AT1-APPLICATIONS BASED QUESTIONS

COURSE CODE: DSA0308

COURSE NAME:NATURAL LANGUAGE AND PREPROCESSING

REG NO:192424322 NAME:M.ARSHINI

ANSWERS:

PROBLEM 1: Resume Information Extraction Using Regular Expressions

A recruitment company receives resumes in text format. It needs a Python program that automatically extracts candidate details and identifies eligible candidates.

The program must:

1. Extract the candidate's name.
2. Identify email addresses.
3. Extract mobile numbers.
4. Detect technical skills such as Python, Java, SQL, Machine Learning, and NLP.
5. Extract years of experience.
6. Generate a structured candidate summary.
7. Display candidates who have at least 2 years of experience

Algorithm:

1. Start the program.
2. Import the re module.
3. Store the resume details in a string.
4. Extract the candidate's name, email, and mobile number using Regex.
5. Identify technical skills such as Python, Java, SQL, Machine Learning, and NLP.
6. Extract the candidate's years of experience.
7. Display all extracted information.
8. Check whether experience is at least 2 years and Python skill is present.

9. Display the eligibility result.
10. Stop the program.

Python Code:

```
import re

resume = """
Name: Ananya Sharma
Email: ananya.sharma@example.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning, NLP
Experience: 3 years
"""

# Extract name, email and mobile
name = re.search(r"Name:\s*(.+)", resume).group(1)
email = re.search(r"\S+@\S+\.\S+", resume).group()
mobile = re.search(r"Mobile:\s*(\d{10})", resume).group(1)

# Detect technical skills
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found = []

for skill in skills:
    if re.search(r"\b" + re.escape(skill) + r"\b",
                 resume, re.I):
        found.append(skill)

# Extract experience
```

```
experience = int(re.search(
    r"Experience:\s*(\d+)", resume
).group(1))

# Display candidate details
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", ", ".join(found))
print("Experience:", experience, "years")

# Check eligibility
if experience >= 2 and "Python" in found:
    print("Eligible for shortlisting")
else:
    print("Not eligible for shortlisting")
```

Input:

Name: Ananya Sharma

Email: ananya.sharma@example.com

Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years

output:

Name: Ananya Sharma

Email: ananya.sharma@example.com

Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years

Eligible for shortlisting

RESUILT:

1.Candidate name extraction

Extracted result: Ananya Sharma

The program locates the Name: label and captures the text following it. This confirms that the name extraction pattern works for the specified input format.

2 . Email identification

Extracted result: ananya.sharma@example.com

The email pattern recognizes the username, @ symbol, domain, and extension. This helps identify candidate contact information for recruitment communication.

3.Mobile number extraction

Extracted result: 9876543210

The pattern captures a ten-digit mobile number following the Mobile: label. It prevents shorter or longer digit sequences from being accepted by this particular pattern.

4.Technical skill detection

Extracted result: Python, SQL, Machine Learning, NLP

The program compares the resume's skills field with a predefined list. Four of the five supported skills are detected; Java is not detected because it does not appear in the input.

5.Experience extraction

Extracted result: 3 years

The program extracts the numeric value 3 and converts it to an integer. This allows the program to compare the candidate's experience against the minimum requirement.

6.Eligibility evaluation

Result: Eligible for shortlisting

The program checks whether experience is at least two years and Python is present. Both conditions are satisfied, so the candidate passes the configured screening criteria.

PROBLEM 2: Product Search System Using Regular Expressions

To develop a Product Search System using Python Regular Expressions (RegEx). The system searches for products based on a given keyword and search type.

The system supports four types of searches:

- Exact Search: Finds a product that exactly matches the keyword.
- Prefix Search: Finds products that start with the keyword.
- Suffix Search: Finds products that end with the keyword.
- Partial Search: Finds products that contain the keyword anywhere in their name.

2. Algorithm

1. Start the program.
2. Import the re module.
3. Create a list containing product names.
4. Define a function to search for products.
5. Accept the search keyword and search type.
6. Create a regular expression pattern based on the search type.
7. Compare the pattern with each product name using re.search().
8. Ignore uppercase and lowercase differences.
9. Display all matching products.
10. Display the total number of matches.
11. Stop the program.

CODE:

```
import re


products = [
    "Apple iPhone",
    "Apple Watch",
    "Samsung Galaxy",
    "Wireless Headphones",
    "Apple AirPods",
    "Gaming Laptop"
```

```
]

def search_products(keyword, search_type):
    escaped_keyword = re.escape(keyword)

    if search_type == "exact":
        pattern = r"^" + escaped_keyword + r"$"
    elif search_type == "prefix":
        pattern = r"^" + escaped_keyword
    elif search_type == "suffix":
        pattern = escaped_keyword + r"$"
    else:
        pattern = escaped_keyword

    matches = [
        product for product in products
        if re.search(pattern, product, re.I)
    ]

    print("\nSearch Type:", search_type)
    print("Keyword:", keyword)
    print("Matching Products:")

    if matches:
        for product in matches:
            print("-", product)
    else:
        print("No products found")

    print("Total Matches:", len(matches))
```

search_products("Apple Watch", "exact")

search_products("Apple", "prefix")

search_products("Laptop", "suffix")

search_products("phone", "partial")

Input

The product list is:

Apple iPhone

Apple Watch

Samsung Galaxy

Wireless Headphones

Apple AirPods

Gaming Laptop

The program searches for four keywords using different search types.

Output

Search type: exact

Keyword: Apple\ Watch

Matching products:

- Apple Watch

Total matches: 1

Search type: prefix

Keyword: Apple

Matching products:

- Apple iPhone

- Apple Watch

- Apple AirPods

Total matches: 3

Search type: suffix

Keyword: Laptop

Matching products:

- Gaming Laptop

Total matches: 1

Search type: partial

Keyword: phone

Matching products:

- Apple iPhone

- Wireless Headphones

Total matches: 2

Note: In this code, keyword is overwritten with its escaped version, so the exact-search heading displays Apple\ Watch. The search itself still works correctly.

c) Result Analysis

Step 1: Product list creation

The program creates a list of six products. This list acts as the product database for the search system.

Step 2: Exact keyword search

The search keyword is Apple Watch.

- Pattern: ^Apple Watch$
- Matching product: Apple Watch
- Total matches: 1

The ^ and $ symbols require the complete product name to match the keyword.

Step 3: Prefix search

The search keyword is Apple.

- Pattern: ^Apple
- Matching products: Apple iPhone, Apple Watch, Apple AirPods
- Total matches: 3

The ^ symbol ensures that the product name starts with the specified keyword.

Step 4: Suffix search

The search keyword is Laptop.

- Pattern: Laptop$
- Matching product: Gaming Laptop
- Total matches: 1

The $ symbol ensures that the product name ends with the specified keyword.

Step 5: Partial keyword search

The search keyword is phone.

- Pattern: phone
- Matching products: Apple iPhone, Wireless Headphones
- Total matches: 2

The keyword can occur anywhere in a product name. Matching is case-insensitive, so phone also matches Phone.

Step 6: Analysis of results

Search type	Number of matches	Analysis
Exact	1	Finds the specified product name
Prefix	3	Finds products beginning with Apple
Suffix	1	Finds products ending with Laptop
Partial	2	Finds products containing phone

Problem 3: University Registration System Using Python Regular Expressions

TO develop a University Registration System using Python Regular Expressions (RegEx). The system validates student registration details, including register number, institutional email, course code, semester, and mobile number.

The system checks whether the entered information follows the required format. If all details are valid, registration is successful; otherwise, the system displays an unsuccessful registration status and identifies the invalid fields.

1. Validate student registration details.
2. Use regular expressions to check different input formats.
3. Reduce errors in student registration.
4. Display the final registration status.

Algorithm

1. Start the program.
2. Import the Python re module.
3. Accept the student's registration details.
4. Validate the register number using a regular expression.
5. Validate the institutional email address.
6. Validate the course code.
7. Check whether the semester is between 1 and 8.
8. Validate the 10-digit mobile number.
9. Display the validation result for each field.
10. If all fields are valid, display Registration Successful; otherwise, display Registration Failed.
11. Stop the program.

CODE:

```
import re


register_no = input("Enter Register Number: ")

email = input("Enter Institutional Email: ")

course = input("Enter Course Code: ")

semester = input("Enter Semester (1-8): ")
```

```
mobile = input("Enter Mobile Number: ")

valid_register = re.fullmatch(r"REG\d{7}", register_no)
valid_email = re.fullmatch(
    r"[A-Za-z0-9._%+-]+@university\.edu", email
)
valid_course = re.fullmatch(r"[A-Z]{2,4}\d{3}", course)
valid_semester = semester.isdigit() and 1 <= int(semester) <= 8
valid_mobile = re.fullmatch(r"[6-9]\d{9}", mobile)

print("\n--- Validation Results ---")
print("Register Number:", "Valid" if valid_register else "Invalid")
print("Institutional Email:", "Valid" if valid_email else "Invalid")
print("Course Code:", "Valid" if valid_course else "Invalid")
print("Semester:", "Valid" if valid_semester else "Invalid")
print("Mobile Number:", "Valid" if valid_mobile else "Invalid")

if all([valid_register, valid_email, valid_course,
        valid_semester, valid_mobile]):
    print("\nRegistration Status: SUCCESSFUL")
else:
    print("\nRegistration Status: FAILED")
```

Input

Register Number

REG2026001

Institutional Email

student@university.edu

Course Code

CS101

Semester

3

Mobile Number

9876543210

Output:

--- Validation Results ---

Register Number: Valid

Institutional Email: Valid

Course Code: Valid

Semester: Valid

Mobile Number: Valid

Registration Status: SUCCESSFUL

Result Analysis

Step 1: Register Number Validation

- Input: REG2026001
- Pattern: REG\d{7}
- Result: Valid

Analysis: The register number begins with REG and contains exactly seven digits after the prefix. Therefore, it satisfies the required format.

Step 2: Institutional Email Validation

- Input: student@university.edu
- Pattern: [A-Za-z0-9._%+-]+@university\.edu
- Result: Valid

Analysis: The email contains a valid username followed by the specified institutional domain, @university.edu. Hence, it passes the validation.

Step 3: Course Code Validation

- Input: CS101
- Pattern: [A-Z]{2,4}\d{3}
- Result: Valid

Analysis: The course code contains two uppercase letters followed by three digits. It matches the specified course code format.

Step 4: Semester Validation

- Input: 3
- Allowed range: 1 to 8
- Result: Valid

Analysis: The semester input is numeric, and its value is within the permitted range of 1–8. Therefore, the semester is valid.

Step 5: Mobile Number Validation

- Input: 9876543210
- Pattern: [6-9]\d{9}
- Result: Valid

Analysis: The mobile number contains ten digits and starts with 9, which falls within the permitted starting digits from 6 to 9. Thus, it passes the format check.

Step 6: Final Registration Status

The program combines the five validation results using the all() function.

- All five fields valid: Registration Successful
- One or more fields invalid: Registration Failed

Analysis: Since all five sample inputs satisfy their respective validation rules, the program displays Registration Status: SUCCESSFUL